

MANUAL DE INSTALACIÓN

Sistema de Gestión de Lavado Vehicular (Vehicle Washing)

Requisitos, configuración, despliegue con Docker, base de datos SQL Server y verificación del servicio

Tipo de manual	Manual de instalación (personal de infraestructura y DevOps)
Integrantes	Brayan Estiven Patiño Cabrera (líder de proyecto) Juan Camilo Guilombo Osorio (líder de frontend) Merari Urbano (líder de backend)
Ficha	3145556
Programa	Tecnólogo en Análisis y Desarrollo de Software (ADSO)
Instructor/a	Karol Daniela Correa
Centro de formación	Industria
Fecha	05 de octubre de 2026
Versión	2.0

Control de versiones

La Tabla 1 registra el historial de cambios del presente documento. Cada vez que el procedimiento de instalación cambie (por ejemplo, por una nueva versión de un servicio o de una dependencia), debe agregarse una fila con la versión, la fecha, la descripción y el responsable.

Tabla 1

Historial de versiones del manual de instalación

Versión	Fecha	Descripción del cambio	Responsable
2.0	05/10/2026	Creación del manual de instalación a partir de la documentación del proyecto (versión del 02/10/2026), dentro de la versión 2.0 del conjunto de manuales.	Equipo desarrollador

Nota. Elaboración propia del equipo desarrollador.

Tabla de contenido

Control de versiones	2
4. Introducción	6
4.1 Fuentes y limitaciones de este documento	6
4.2 Alcance	6
5. Objetivo	8
5.1 Objetivo general	8
5.2 Objetivos específicos	8
6. Requisitos mínimos de hardware	9
6.1 Sistemas operativos compatibles	9
6.2 Puertos de red	10
7. Software requerido	11
7.1 Comprobación de las herramientas instaladas	11
8. Descarga u obtención del proyecto	13
8.1 Clonación de los repositorios	13
8.2 Estructura esperada	14
9. Instalación de dependencias	15
9.1 Imágenes de contenedores	15
9.2 Dependencias del backend	15
9.3 Dependencias de la aplicación web	15
9.4 Dependencias de la aplicación móvil	15
9.5 Cliente de migraciones y de SQL	15
10. Configuración de variables de entorno	17
10.1 Creación del archivo .env	17
10.2 Variables confirmadas por la documentación	17
10.3 Variables que el entorno necesita	18
10.4 Generación de secretos	19
10.5 Ejemplo de estructura del archivo .env	19
10.6 Configuración de las aplicaciones cliente	20
11. Creación e importación de base de datos	21
11.1 Inicio de SQL Server	21

11.2 Creación de la base de datos	22
11.3 Aplicación de las migraciones con Liquibase (método recomendado)	22
11.4 Método alternativo: script único	23
11.5 Usuarios de base de datos por servicio	24
11.6 Verificación de la estructura y de los datos iniciales	24
11.7 Respaldo y reinicio de la base local	25
12. Ejecución del backend	26
12.1 Orden de arranque	26
12.2 Modo mínimo sin mensajería	26
12.3 Despliegue con Docker Compose (método recomendado)	26
12.4 Ejecución de un servicio fuera de Docker (alternativa de desarrollo)	27
12.5 Configuración de RabbitMQ y de correo	28
12.6 Registros, detención y actualización	28
12.7 Almacenamiento de comprobantes de pago	28
13. Ejecución del frontend	29
13.1 Aplicación web (Angular)	29
13.2 Aplicación móvil (React Native con Expo)	29
13.3 Maquetas estáticas (opcional)	30
14. Acceso al sistema	31
14.1 Primer acceso a la aplicación web	31
14.2 Interfaces de apoyo	32
15. Usuario de prueba	33
15.1 Reglas para las cuentas	33
15.2 Creación de un cliente de prueba	33
15.3 Cuentas de operario y de administrador	33
16. Verificación de instalación	35
16.1 Infraestructura y base de datos	35
16.2 Mensajería y correo	36
16.3 API Gateway	36
16.4 Flujo funcional mínimo	36
16.5 Verificación de la auditoría	37
17. Errores comunes y soluciones	38

18. Glosario	41
19. Anexos	42
Anexo A. Lista de figuras pendientes de captura	42
Anexo B. Información por confirmar antes de la entrega	42
Anexo C. Resumen de comandos de instalación	44
Anexo D. Lista de chequeo de seguridad previa a compartir el entorno	44
Anexo E. Lista de chequeo de entrega	45
Referencias	46

4. Introducción

El Sistema de Gestión de Lavado Vehicular (Vehicle Washing) es una plataforma compuesta por una puerta de enlace (API Gateway), seis microservicios de backend desarrollados con Java y Spring Boot, una base de datos Microsoft SQL Server, un bróker de mensajes RabbitMQ, un servidor SMTP de desarrollo (Mailpit) y dos clientes: una aplicación web en Angular y una aplicación Android en React Native con Expo. El único entorno que la documentación del proyecto declara dentro de su alcance es el local, ejecutado con Docker; no existen entornos de pruebas integradas (staging) ni de producción.

Este manual explica cómo preparar un equipo, instalar sus dependencias, configurar las variables de entorno, crear y poblar la base de datos, iniciar cada componente en el orden correcto y comprobar que el sistema funciona. Está dirigido al personal de infraestructura y de DevOps, o a cualquier persona que deba poner en marcha el sistema en un equipo local o en un servidor de desarrollo. Se parte de la premisa, común en las guías de despliegue continuo, de que una instalación debe ser repetible, automatizable y verificable (Humble y Farley, 2010), y de que la configuración que cambia entre entornos debe residir en el entorno y no en el código (Wiggins, 2017).

4.1 Fuentes y limitaciones de este documento

El manual se elaboró a partir del repositorio de documentación del proyecto (arquitectura, ADR, modelo de datos, estrategia de migración y políticas de seguridad). **Dicho material no incluye el código fuente de los servicios, ni el repositorio de infraestructura (*lavarapido-infra*) con su archivo de Docker Compose y su archivo *.env.example* reales.** Además, el documento de configuración local del repositorio (*local-setup.md*) es una plantilla sin adaptar (menciona PostgreSQL, Redis y puertos genéricos), por lo que no se utilizó como fuente de valores.

En consecuencia, este manual distingue dos tipos de información. La primera es la **confirmada** por la documentación: componentes, orden de arranque, esquemas de base de datos, orden de las migraciones, nombres de variables como *MESSAGING_ENABLED* o *CUSTOMER_SERVICE_URL*, y reglas de seguridad. La segunda es la que **depende del repositorio de infraestructura** (nombres de archivos, de servicios de Compose, puertos y nombres de otras variables): se presenta con marcadores entre ángulos, por ejemplo *<PUERTO_GATEWAY>*, o con valores por defecto de las imágenes oficiales, y se lista en el Anexo B para que el equipo la confirme antes de la entrega. Ningún marcador debe copiarse literalmente a un terminal.

4.2 Alcance

Cubre: instalación local con Docker de SQL Server, RabbitMQ, Mailpit, los seis servicios y el gateway; creación de la base de datos y aplicación de migraciones con Liquibase; ejecución de la aplicación web Angular y de la aplicación Android; accesos iniciales, usuarios de prueba, verificación y solución de errores comunes.

No cubre: despliegue en staging o producción (fuera del alcance académico), el pipeline de integración continua con Jenkins (que pertenece a la sección de DevOps del repositorio de documentación), la publicación de la aplicación móvil en tiendas, ni el uso funcional del sistema, que se explica en el Manual de usuario y en el Manual de administrador.

5. Objetivo

5.1 Objetivo general

Permitir que una persona de infraestructura o DevOps instale y ponga en funcionamiento el Sistema de Gestión de Lavado Vehicular siguiendo instrucciones claras, completas y verificables.

5.2 Objetivos específicos

- Definir los requisitos de hardware, sistema operativo, software y red necesarios para ejecutar el sistema completo.
- Describir cómo obtener el código y las dependencias de backend, web y móvil.
- Explicar la configuración de las variables de entorno y las prácticas para manejar secretos.
- Detallar la creación de la base de datos de nueve esquemas, la aplicación de las migraciones y la carga de datos iniciales.
- Indicar el orden de arranque de los componentes y los comandos para iniciarlos, detenerlos y revisar sus registros.
- Ofrecer pruebas rápidas de verificación y una guía de solución de errores frecuentes.

6. Requisitos mínimos de hardware

La documentación del proyecto no fija requisitos de hardware. Los valores de la Tabla 2 son recomendaciones de este manual, calculadas para ejecutar simultáneamente nueve contenedores de aplicación (gateway y seis servicios de Java, SQL Server y RabbitMQ), más Mailpit y las herramientas de desarrollo. El único mínimo con respaldo oficial es el de SQL Server en contenedor, que exige al menos 2 GB de memoria RAM disponibles para el contenedor (Microsoft, s. f.). En Docker Desktop para Windows y macOS, esa memoria debe asignarse explícitamente a la máquina virtual de Docker.

Tabla 2
Requisitos de hardware recomendados

Recurso	Mínimo	Recomendado	Observación
Procesador	4 núcleos, 64 bits (amd64)	6 u 8 núcleos	La imagen oficial de SQL Server es para amd64; en equipos con procesador ARM se ejecuta mediante emulación.
Memoria RAM	8 GB	16 GB	Con 8 GB, asignar al menos 5 GB a Docker y cerrar el resto de aplicaciones.
Memoria para Docker	5 GB	8 GB o más	SQL Server requiere 2 GB como mínimo; cada servicio Java consume memoria adicional.
Disco libre	20 GB	40 GB en SSD	Imágenes de Docker, volúmenes de datos, dependencias de Maven y npm y registros.
Red	Acceso a internet	Banda ancha estable	Descarga de imágenes y dependencias; para pruebas en teléfono físico, la misma red local.

Nota. Recomendaciones de este manual. El requisito de 2 GB de RAM para SQL Server es el publicado por Microsoft (s. f.). Se recomienda validar los valores de la tabla con una instalación real y ajustarlos.

6.1 Sistemas operativos compatibles

Tabla 3
Sistemas operativos y requisitos adicionales

Sistema operativo	Requisito adicional
Windows 10 o 11 (64 bits)	Docker Desktop con WSL 2 habilitado y virtualización activada en el BIOS o UEFI.
Ubuntu 22.04 LTS o superior	Docker Engine 24 o superior con el complemento Docker Compose 2.20 o superior.
macOS	Docker Desktop; con procesador Apple Silicon, habilitar la emulación de amd64 para SQL Server.

Nota. Elaboración propia con base en las versiones mínimas de Docker de la documentación del proyecto (Docker 24+ y Compose 2.20+).

6.2 Puertos de red

La documentación indica que los puertos de los servicios de aplicación son un detalle de configuración que debe tomarse de la configuración activa de Docker y no inventarse. La Tabla 4 reúne los puertos por defecto de las imágenes y herramientas estándar; el puerto de cada servicio de aplicación debe confirmarse en el archivo de Compose.

Tabla 4
Puertos que deben estar libres en el equipo anfitrión

Componente	Puerto	Origen del valor
SQL Server	1433	Valor por defecto de la imagen oficial de SQL Server.
RabbitMQ (AMQP)	5672	Valor por defecto de RabbitMQ.
RabbitMQ (consola de administración)	15672	Valor por defecto del complemento de administración.
Mailpit (SMTP)	1025	Valor por defecto de Mailpit.
Mailpit (interfaz web)	8025	Valor por defecto de Mailpit.
API Gateway	8080	Valor de la plantilla de configuración local; confirmar.
Seis servicios de dominio	<PUERTO_SERVICIO>	No documentado; tomar del archivo de Compose.
Aplicación web (servidor de desarrollo de Angular)	4200	Valor por defecto de Angular CLI.
Aplicación móvil (Metro de Expo)	8081	Valor por defecto de Expo.

Nota. Los valores marcados como por defecto pueden haberse cambiado en el repositorio de infraestructura; ver Anexo B.

7. Software requerido

La Tabla 5 lista las herramientas necesarias. Las versiones de Java y de Docker proceden de la documentación; las versiones de Node.js, Angular, Expo y las herramientas de construcción deben tomarse de los archivos *package.json*, *pom.xml* o equivalentes de cada repositorio, donde la política del proyecto exige fijar versiones exactas.

Tabla 5
Software requerido para la instalación

Herramienta	Versión	Uso
Git	Reciente	Obtener los repositorios del proyecto.
Docker Engine o Docker Desktop	24.0 o superior	Ejecutar SQL Server, RabbitMQ, Mailpit y los servicios.
Docker Compose	2.20 o superior (comando <i>docker compose</i>)	Definir y levantar el entorno local completo.
JDK	21 (LTS)	Compilar y ejecutar los servicios fuera de Docker.
Maven (o Gradle, según el repositorio)	Según el repositorio	Construir los servicios y revisar dependencias.
Node.js y npm	Versión LTS indicada por el repositorio	Aplicación web Angular y aplicación móvil.
Angular CLI	Según <i>package.json</i>	Servidor de desarrollo y compilación de la aplicación web.
Expo CLI y EAS CLI (mediante <i>npm</i>)	Según <i>package.json</i>	Ejecutar la aplicación móvil y compilar versiones con notificaciones push.
Liquibase CLI	4.x	Aplicar las migraciones de base de datos.
sqlcmd	Reciente	Ejecutar sentencias SQL y verificar la base de datos desde la terminal.
Cliente SQL gráfico (opcional)	SSMS o Azure Data Studio	Inspección visual de la base de datos.
curl	Reciente	Pruebas de verificación del servicio.
Android Studio o un teléfono Android (opcional)	Reciente	Emulador o dispositivo para la aplicación móvil.
Cliente HTTP (opcional)	Postman o extensión REST Client de VS Code	Ejecutar el flujo de demostración <i>http/flujo-demo.http</i> .

Nota. Elaboración propia. Docker 24+ y Compose 2.20+ proceden de la documentación; JDK 21 de la guía del stack Java/Spring; Liquibase de la estrategia de migración.

7.1 Comprobación de las herramientas instaladas

Antes de continuar, se ejecutan los comandos siguientes en una terminal. Cada uno debe imprimir una versión; si alguno falla, se instala la herramienta correspondiente.

```
git --version
docker --version
docker compose version
java -version
mvn -version
node -v
npm -v
sqlcmd -?
liquibase --version
curl --version
```

La salida de *java -version* debe indicar la versión 21. Con una versión anterior, los servicios fallan al iniciar con el error *UnsupportedClassVersionError* (sección 17). En Windows, *docker info* debe mostrar el motor de Linux (WSL 2) y no el de contenedores de Windows.

Figura 1

Verificación de versiones de las herramientas

[ESPACIO RESERVADO PARA LA CAPTURA]

Ubicación/Pantalla: Terminal del equipo de instalación (PowerShell, Bash o Zsh).

Descripción exacta de la captura: Salida de los comandos de la sección 7.1 con las versiones de Git, Docker, Docker Compose, Java 21, Maven, Node.js, npm y Liquibase visibles en una sola captura.

Nota. Sirve de evidencia del entorno preparado.

8. Descarga u obtención del proyecto

El proyecto se distribuye en varios repositorios de Git. La Tabla 6 reúne los repositorios que la documentación y el equipo mencionan. Las direcciones exactas deben tomarse del catálogo de servicios o del repositorio de GitHub de la organización, porque el material analizado no las contiene.

Tabla 6
Repositorios del proyecto

Repositorio	Contenido	Dirección
lavarapido-infra	Definición de Docker Compose del entorno local y el archivo <i>.env.example</i> .	<URL_REPOSITORIO_INFRA>
lavarapido-backend	Código de los seis servicios y del gateway; solicitudes de demostración en <i>http/flujo-demo.http</i> .	<URL_REPOSITORIO_BACKEND>
Scripts y migraciones de base de datos	Changelog maestro de Liquibase (<i>db/changelog/db.changelog-master.xml</i>) y script único <i>lavarapido-6-services-sqlserver.sql</i> .	<URL_O_RUTA_DE_LA_BASE_DE_DATOS>
Front-end-proyecto-web	Aplicación web Angular.	<URL_REPOSITORIO_WEB>
LavadoVehicular-Movil	Aplicación Android (React Native con Expo).	<URL_REPOSITORIO_MOVIL>
Repositorio de documentación	Arquitectura, ADR, modelo de datos y maquetas HTML.	<URL_REPOSITORIO_DOCUMENTACION>

Nota. Los nombres de repositorio proceden de la documentación (*lavarapido-infra*, *lavarapido-backend*) y del equipo. El repositorio donde residen los changelogs de Liquibase debe confirmarse (Anexo B).

8.1 Clonación de los repositorios

Se crea un directorio de trabajo común y se clona cada repositorio. Las convenciones del proyecto usan la rama *develop* para integración y la rama *main* para versiones estables; para una instalación de evaluación se recomienda *main* o la etiqueta de la versión entregada.

```
mkdir -p ~/lavarapido && cd ~/lavarapido
git clone <URL_REPOSITORIO_INFRA> lavarapido-infra
git clone <URL_REPOSITORIO_BACKEND> lavarapido-backend
git clone <URL_REPOSITORIO_WEB> Front-end-proyecto-web
git clone <URL_REPOSITORIO_MOVIL> LavadoVehicular-Movil
```

Para fijar una versión concreta se consultan las etiquetas disponibles y se cambia a la elegida en cada repositorio:

```
cd lavarapido-backend
git fetch --tags
git tag -l
git checkout <ETIQUETA_O_RAMAS> # por ejemplo: main
cd ..
```

En PowerShell, el directorio de trabajo se crea con *New-Item -ItemType Directory -Force \$HOME\lavarapido*; los comandos de *git* son idénticos.

8.2 Estructura esperada

Al finalizar, el directorio de trabajo debe contener los repositorios clonados. Dentro de *lavarapido-infra* deben existir, como mínimo, el archivo de Compose y el archivo *.env.example*; si no están, se está en una rama equivocada.

```
lavarapido/
|-- lavarapido-infra/      (docker compose + .env.example)
|-- lavarapido-backend/   (seis servicios + gateway + http/flujo-demo.http)
|-- Front-end-proyecto-web/ (Angular)
`-- LavadoVehicular-Movil/ (React Native / Expo)
```

Figura 2

Repositorios clonados en el directorio de trabajo

[ESPACIO RESERVADO PARA LA CAPTURA]

Ubicación/Pantalla: Terminal del equipo de instalación, dentro del directorio de trabajo.

Descripción exacta de la captura: Salida de un listado del directorio con los cuatro repositorios y, dentro de *lavarapido-infra*, el archivo de Compose y el archivo *.env.example*.

9. Instalación de dependencias

9.1 Imágenes de contenedores

El entorno local usa imágenes oficiales de SQL Server (mcr.microsoft.com/mssql/server), RabbitMQ con consola de administración y Mailpit, además de las imágenes de los servicios, que se construyen a partir del código. Desde el directorio de infraestructura se descargan las imágenes base y se valida la sintaxis del archivo de Compose:

```
cd ~/lavarapido/lavarapido-infra
docker compose config --quiet && echo "Compose valido"
docker compose pull
```

El comando *docker compose config* falla si faltan variables obligatorias; en ese caso se completa primero el archivo *.env* (sección 10) y se vuelve a ejecutar. SQL Server admite las etiquetas *2019-latest*, *2022-latest* y *2025-latest*; la documentación del proyecto exige SQL Server 2016 o superior (Microsoft, s. f.).

9.2 Dependencias del backend

Los servicios usan Java 21 y Spring Boot. Si el archivo de Compose construye las imágenes de los servicios desde sus Dockerfiles, las dependencias se descargan durante la construcción y este paso puede omitirse. Para compilar y ejecutar los servicios sin Docker, se descargan las dependencias con Maven:

```
cd ~/lavarapido/lavarapido-backend
mvn -B -DskipTests clean package
```

Si el repositorio usa Gradle en lugar de Maven, el equivalente es *./gradlew build -x test*. La política de seguridad del proyecto exige versiones fijas, sin versiones flotantes, y revisar las dependencias desactualizadas antes de cada entrega:

```
mvn versions:display-dependency-updates
```

9.3 Dependencias de la aplicación web

```
cd ~/lavarapido/Front-end-proyecto-web
npm ci
```

El comando *npm ci* instala exactamente las versiones del archivo *package-lock.json* y falla si este no coincide con *package.json*, lo que garantiza instalaciones repetibles. Solo si el repositorio no incluye *package-lock.json* se usa *npm install*. Se revisan además las vulnerabilidades conocidas con *npm audit*.

9.4 Dependencias de la aplicación móvil

```
cd ~/lavarapido/LavadoVehicular-Movil
npm ci
```

9.5 Cliente de migraciones y de SQL

Liquibase y sqlcmd se instalan en el equipo desde los instaladores oficiales de cada herramienta. Una alternativa que evita instalaciones locales es ejecutarlos desde contenedores; en ese caso, los comandos de la sección 11 se anteponen con *docker run* o *docker compose run* y se monta el directorio de los changelogs.

Figura 3

Construcción satisfactoria del backend

[ESPACIO RESERVADO PARA LA CAPTURA]

Ubicación/Pantalla: Terminal, directorio lavarapido-backend.

Descripción exacta de la captura: Salida final de mvn clean package con el mensaje BUILD SUCCESS y la lista de módulos construidos (gateway y seis servicios).

Nota. Solo aplica si se compilan los servicios fuera de Docker.

10. Configuración de variables de entorno

Los valores que cambian entre entornos (cadena de conexión, credenciales, secreto del JWT, configuración de correo y puertos) deben residir fuera del código fuente (Wiggins, 2017). La documentación del proyecto fija esa regla: los secretos se leen de variables de entorno desde un archivo `.env` excluido de Git, y el repositorio solo contiene el archivo `.env.example` con los nombres de las variables y valores de ejemplo. Los secretos reales no deben confirmarse en el repositorio ni publicarse en este manual.

10.1 Creación del archivo `.env`

```
cd ~/lavarapido/lavarapido-infra
cp .env.example .env
chmod 600 .env                # solo en Linux y macOS
git check-ignore -v .env      # debe mostrar la regla de .gitignore que lo excluye
```

En PowerShell, la copia se hace con *Copy-Item .env.example .env*. Si el comando *git check-ignore* no imprime nada, el archivo `.env` no está excluido y no debe llenarse con secretos hasta corregir el archivo `.gitignore`. Para revisar que ningún secreto quedó pendiente de confirmación, se ejecuta *git status* y se comprueba que `.env` no aparezca como archivo nuevo.

10.2 Variables confirmadas por la documentación

Tabla 7**Variables de configuración citadas por la documentación**

Variable	Servicio	Efecto y valor recomendado en local
CUSTOMER_SERVICE_URL	booking-service	Dirección base de customer-service, usada para resolver y validar vehículos. En la red de Docker, el nombre del servicio y su puerto interno.
CUSTOMER_SERVICE_TIMEOUT	booking-service	Tiempo de espera de esa llamada. Si se agota, la reserva responde 503 CUSTOMER_SERVICE_UNAVAILABLE.
BOOKING_ZONE	booking-service	Zona horaria de las horas reservables. Para Colombia corresponde el identificador <i>America/Bogota</i> ; confirmar el formato esperado.
MESSAGING_ENABLED	security-service y notification-service	Activa RabbitMQ (publicador y consumidor). Por defecto <i>false</i> . Para probar el flujo completo de notificaciones, <i>true</i> .
PUSH_ENABLED	notification-service	Activa el envío real a la API de Expo. Por defecto <i>false</i> : solo se registra. Requiere una compilación EAS de la aplicación móvil.
MAIL_ENABLED	security-service y notification-service	Activa el envío de correo (bienvenida y código de recuperación). Con <i>false</i> , no se envía correo.
MAIL_*	security-service y notification-service	Servidor, puerto y credenciales SMTP. En local, Mailpit (puerto 1025 por defecto, sin autenticación); también se admite una cuenta de Gmail con contraseña de aplicación.

Nota. Fuente: ADR-011, ADR-012 y deployment.md. Los nombres exactos de las variables *MAIL_** deben tomarse del archivo *.env.example* real.

10.3 Variables que el entorno necesita

Además de las anteriores, la documentación enumera categorías de configuración que todo entorno requiere, sin fijar sus nombres. Los nombres reales deben copiarse del archivo *.env.example* de *lavarapido-infra*.

Tabla 8

Categorías de configuración que deben definirse

Categoría	Qué debe definirse	Observación
Conexión a SQL Server	Servidor, puerto, nombre de la base de datos y un usuario por servicio con derechos solo sobre sus esquemas.	Contraseña robusta; nunca la cuenta sa en los servicios.
Contraseña de sa	Contraseña del administrador del contenedor de SQL Server.	Debe cumplir la política de contraseñas fuertes de SQL Server; si no, el contenedor no inicia.
Secreto del JWT	Clave de firma HS256 compartida por los servicios que validan el token.	Mínimo 32 caracteres; idéntica en todos los servicios.
RabbitMQ	Servidor, puerto AMQP, usuario y contraseña.	Solo necesarias si MESSAGING_ENABLED es true.
SMTP	Servidor y puerto del correo.	Ver variables MAIL_.*.
Puertos de aplicación	Puerto del gateway y de cada servicio.	No están fijados en la documentación.
Origen permitido (CORS)	Dirección desde la que el navegador llama al gateway.	Debe incluir la dirección de la aplicación web.

Nota. Fuente: deployment.md y security-rules.md.

10.4 Generación de secretos

El secreto del JWT y las contraseñas deben generarse con un generador criptográficamente seguro y no escribirse a mano. Con OpenSSL (Linux, macOS o Git Bash en Windows):

```
openssl rand -base64 48          # secreto del JWT (mayor a 32 caracteres)
openssl rand -base64 24          # contraseña de servicio de base de datos
```

En PowerShell, un equivalente es `[Convert]::ToBase64String((1..48 | ForEach-Object { Get-Random -Maximum 256 }) -as [byte[]])`, aunque se prefiere `openssl` por usar una fuente aleatoria criptográfica. El secreto generado se pega en el archivo `.env`, nunca en un mensaje, un documento compartido ni un repositorio.

10.5 Ejemplo de estructura del archivo .env

El fragmento siguiente muestra únicamente la **estructura**. Los nombres de las variables marcadas como ilustrativas deben sustituirse por los del archivo `.env.example` real, y los valores entre ángulos por los propios del entorno. Los secretos se representan con asteriscos y nunca se publican.

```
# --- Base de datos (nombres ILUSTRATIVOS) ---
DB_HOST=sqlserver
DB_PORT=1433
DB_NAME=<NOMBRE_BASE_DE_DATOS>
```

```

MSSQL_SA_PASSWORD=*****
DB_USER_<SERVICIO>=<usuario_del_servicio>
DB_PASSWORD_<SERVICIO>=*****

# --- Seguridad (nombre ILUSTRATIVO) ---
JWT_SECRET=*****

# --- Variables confirmadas por la documentacion ---
CUSTOMER_SERVICE_URL=http://customer-service:<PUERTO_SERVICIO>
CUSTOMER_SERVICE_TIMEOUT=<TIEMPO_DE_ESPERA>
BOOKING_ZONE=America/Bogota
MESSAGING_ENABLED=true
PUSH_ENABLED=false
MAIL_ENABLED=true
# MAIL_* -> servidor mailpit, puerto 1025 (nombres segun .env.example)

# --- RabbitMQ (nombres ILUSTRATIVOS) ---
RABBITMQ_HOST=rabbitmq
RABBITMQ_USER=<usuario>
RABBITMQ_PASSWORD=*****

```

Nota. Estructura ilustrativa elaborada por este manual. No sustituye al archivo `.env.example` del repositorio de infraestructura.

10.6 Configuración de las aplicaciones cliente

Las dos aplicaciones cliente solo necesitan conocer la dirección base del API Gateway, que es su único punto de entrada. El archivo donde se declara (por ejemplo, un archivo de entorno de Angular o una variable de Expo) debe tomarse del repositorio de cada cliente. Para la aplicación móvil, la dirección depende del dispositivo: en el emulador de Android, el equipo anfitrión se alcanza con `10.0.2.2` y no con `localhost`; en un teléfono físico se usa la dirección IP local del equipo anfitrión dentro de la misma red.

Figura 4

Archivo `.env` configurado (con secretos ocultos)

[ESPACIO RESERVADO PARA LA CAPTURA]

Ubicación/Pantalla: Editor de texto, archivo `lavarapido-infra/.env`.

Descripción exacta de la captura: Contenido del archivo con todas las variables diligenciadas; las contraseñas, el secreto del JWT y cualquier credencial deben aparecer cubiertos con asteriscos.

Nota. No capturar valores reales de secretos.

11. Creación e importación de base de datos

El sistema usa una única base de datos de Microsoft SQL Server (versión 2016 o superior) dividida en nueve esquemas: *security*, *audit*, *customer*, *catalog*, *booking*, *execution*, *promotion*, *payment* y *notification*. La estructura se crea con Liquibase mediante un único changelog maestro ordenado para toda la base de datos, no uno por servicio. El proyecto también conserva un script único de SQL (*lavarapido-6-services-sqlserver.sql*) pensado para ejecutarse una sola vez sobre una base vacía.

11.1 Inicio de SQL Server

Con el archivo de Compose, SQL Server se levanta como cualquier otro servicio de infraestructura. El nombre del servicio (<SERVICIO_SQLSERVER>) debe tomarse del archivo de Compose; *docker compose config --services* lista los nombres disponibles.

```
cd ~/lavarapido/lavarapido-infra
docker compose config --services
docker compose up -d <SERVICIO_SQLSERVER>
docker compose ps
docker compose logs --tail 40 <SERVICIO_SQLSERVER>
```

El estado debe ser *running* (o *healthy*, si hay verificación de salud) y el registro debe mostrar que SQL Server está listo para aceptar conexiones. Si el contenedor se detiene a los pocos segundos, las causas habituales son una contraseña de sa que no cumple la política, memoria insuficiente (menos de 2 GB) o no haber aceptado la licencia (*ACCEPT_EULA=Y*).

Si el entorno no usa Compose para SQL Server, se puede iniciar el contenedor oficial de forma independiente (Microsoft, s. f.):

```
docker run -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=<CONTRASENA_SA>" \
  -p 1433:1433 --name lavarapido-sql \
  -v lavarapido-sql-data:/var/opt/mssql \
  -d mcr.microsoft.com/mssql/server:2022-latest
```

El volumen nombrado (*lavarapido-sql-data*) conserva los datos cuando el contenedor se recrea. Se comprueba la conectividad desde el equipo anfitrión; la opción *-C* acepta el certificado autofirmado del contenedor:

```
sqlcmd -S localhost,1433 -U sa -P "<CONTRASENA_SA>" -C -Q "SELECT @@VERSION"
```

Figura 5

SQL Server en ejecución y conexión satisfactoria

[ESPACIO RESERVADO PARA LA CAPTURA]

Ubicación/Pantalla: Terminal, lavarapido-infra.

Descripción exacta de la captura: Salida de *docker compose ps* con el contenedor de SQL Server en estado *running/healthy* y, debajo, el resultado de *SELECT @@VERSION* desde *sqlcmd*.

11.2 Creación de la base de datos

El nombre de la base de datos no está fijado en la documentación; se usa el marcador `<NOMBRE_BASE_DE_DATOS>` (abreviado `<BD>` en algunos comandos), que debe ser el mismo valor configurado en el archivo `.env`.

```
sqlcmd -S localhost,1433 -U sa -P "<CONTRASENA_SA>" -C -Q \
"IF DB_ID(N'<BD>') IS NULL CREATE DATABASE [<BD>];"
```

11.3 Aplicación de las migraciones con Liquibase (método recomendado)

La estrategia de migración del proyecto utiliza archivos SQL con el encabezado `--liquibase formatted sql`, organizados bajo `db/changelog/` y enlazados desde el archivo raíz `db.changelog-master.xml`. Liquibase ejecuta los archivos en el orden en que el maestro los incluye, no en orden alfabético, y registra lo ejecutado en las tablas `DATABASECHANGELOG` y `DATABASECHANGELOGLOCK`.

Tabla 9
Orden de ejecución de los changelogs

Archivo	Contenido
001-create-schemas	Los nueve esquemas y un rol de base de datos por servicio con derechos solo sobre sus esquemas.
002-create-security-and-audit-core	person, app_user, role, permission, role_permission, user_role, user_session, password_reset_token y audit_log.
003 a 010	user_preference y los esquemas customer, catalog, booking, execution, promotion, payment y notification.
011-create-overlap-triggers	Disparadores contra superposición de turnos.
012-create-service-price-immutability-trigger	Inmutabilidad del precio.
013-create-audit-triggers	Genera <code>tr_touch_<tabla></code> para toda tabla con <code>row_version</code> ; debe ejecutarse al final de los esquemas.
seed/101 a 109	Datos iniciales: roles, catálogos de vehículo, categorías de servicio, estados de reserva, estados de ejecución, tipos de descuento, catálogos de pago, tipos de notificación, bahías y horarios.

Nota. Fuente: 06-data/migration-strategy.md. La numeración puede haber evolucionado (por ejemplo, las ADR-010 y ADR-011 citan changesets 016 a 018).

Paso 1. Archivo de propiedades. Se crea un archivo `liquibase.properties` junto al directorio `db`, sin guardar en él la contraseña (se entrega por variable de entorno al ejecutar):

```
changeLogFile=db/changelog/db.changelog-master.xml
url=jdbc:sqlserver://localhost:1433;databaseName=<NOMBRE_BASE_DE_DATOS>;\
encrypt=true;trustServerCertificate=true
username=<USUARIO_CON_DERECHOS_DE_DDL>
```

El usuario debe poder crear esquemas, tablas, roles y disparadores. En un entorno local descartable puede usarse `sa`; en cualquier entorno compartido debe usarse una cuenta de migración dedicada. La opción `trustServerCertificate=true` es aceptable únicamente con el certificado autofirmado de un contenedor local.

Paso 2. Validar y previsualizar. Se comprueba que el changelog sea válido y se revisa el SQL que se ejecutará, sin modificar la base:

```
export LIQUIBASE_COMMAND_PASSWORD='<CONTRASENA>'
liquibase validate
liquibase status --verbose
liquibase update-sql > plan-migracion.sql
```

En PowerShell, la variable se asigna con `$env:LIQUIBASE_COMMAND_PASSWORD = '<CONTRASENA>'`. El archivo `plan-migracion.sql` debe revisarse antes del paso siguiente.

Paso 3. Aplicar.

```
liquibase update
liquibase history
```

liquibase history debe listar los changesets ejecutados, desde la creación de esquemas hasta los datos iniciales. Si *update* termina sin errores, la base queda lista; si falla, no se edita el changeset fallido: se corrige la causa (sección 17) o, en un entorno local descartable, se recrea la base (sección 11.7).

Reglas que deben respetarse siempre. Los changesets son siempre hacia adelante, sin bloques de reversión. Nunca se edita un changeset ya ejecutado: Liquibase calcula una suma de verificación de cada uno y se niega a continuar si el contenido cambió, lo cual es una protección intencional y no un error que se deba evadir. Los datos iniciales usan `SET IDENTITY_INSERT` con identificadores fijos, y dos de ellos son críticos: `booking_status_id = 5` (CANCELLED) está escrito dentro de la restricción `ck_booking_cancel` y del disparador `tr_booking_bay_overlap`, y `payment_status_id = 3` (APPROVED) está escrito dentro del índice `ux_payment_one_approved_per_booking`. Resembrar con otro orden de filas rompería esas reglas sin generar un error visible.

Figura 6

Resultado de liquibase update y liquibase history

[ESPACIO RESERVADO PARA LA CAPTURA]

Ubicación/Pantalla: Terminal, directorio del changelog.

Descripción exacta de la captura: Salida de `liquibase update` indicando el número de changesets ejecutados y la terminación exitosa, seguida de la lista de `liquibase history` con los changesets 001 a 013 y los datos iniciales.

11.4 Método alternativo: script único

Para una base vacía, la alternativa es ejecutar el script único del proyecto con *sqlcmd*. Este método **no debe combinarse** con Liquibase sobre la misma base: Liquibase intentaría crear de nuevo objetos que ya existen. El script único tampoco deja registro de lo ejecutado.

```
sqlcmd -S localhost,1433 -U sa -P "<CONTRASENA_SA>" -C -d <NOMBRE_BASE_DE_DATOS> \
-b -i lavarapido-6-services-sqlserver.sql
```

La opción *-b* detiene la ejecución con un código de salida distinto de cero ante el primer error. Según la ADR-011, este script debe actualizarse para incluir las tablas *notification.device_token* y *notification.processed_event*; si no las incluye, el servicio de notificaciones no podrá registrar dispositivos ni eliminar eventos duplicados, y es preferible el método con Liquibase.

11.5 Usuarios de base de datos por servicio

La política de seguridad indica que cada servicio usa un inicio de sesión de SQL Server con derechos únicamente sobre sus propios esquemas, y que el changeset 001 crea un rol de base de datos por servicio. La documentación no indica cómo se crean los inicios de sesión ni qué nombres tienen los roles. La estructura siguiente muestra el procedimiento genérico; los nombres de rol y de usuario deben tomarse del changeset 001.

```
-- Ejecutar con un usuario administrador, repitiendo para cada servicio
CREATE LOGIN [<LOGIN_SERVICIO>]
  WITH PASSWORD = '<CONTRASENA_ROBUSTA>', CHECK_POLICY = ON;
USE [<NOMBRE_BASE_DE_DATOS>];
CREATE USER [<LOGIN_SERVICIO>] FOR LOGIN [<LOGIN_SERVICIO>];
ALTER ROLE [<ROL_DEL_SERVICIO>] ADD MEMBER [<LOGIN_SERVICIO>];
```

Nota. Procedimiento genérico de SQL Server; los nombres de rol deben confirmarse en el changeset 001 (Anexo B).

11.6 Verificación de la estructura y de los datos iniciales

Las consultas siguientes comprueban que la base quedó completa. Se ejecutan con *sqlcmd -d <NOMBRE_BASE_DE_DATOS>* o desde un cliente gráfico.

```
-- 1) Los nueve esquemas deben aparecer
SELECT name FROM sys.schemas
WHERE name IN ('security','audit','customer','catalog','booking',
              'execution','promotion','payment','notification')
ORDER BY name;

-- 2) Tablas por esquema (la documentacion describe 44; las ADR-010/011 suman 5 mas)
SELECT s.name AS esquema, COUNT(*) AS tablas
FROM sys.tables t JOIN sys.schemas s ON s.schema_id = t.schema_id
WHERE s.name IN ('security','audit','customer','catalog','booking',
              'execution','promotion','payment','notification')
GROUP BY s.name ORDER BY s.name;

-- 3) Cada tabla con row_version debe tener su disparador tr_touch_
SELECT (SELECT COUNT(DISTINCT c.object_id) FROM sys.columns c
        JOIN sys.tables t ON t.object_id = c.object_id
        WHERE c.name = 'row_version') AS tablas_con_row_version,
       (SELECT COUNT(*) FROM sys.triggers
        WHERE name LIKE 'tr[_]touch[_]%%') AS disparadores;
```

```
-- 4) Identificadores fijos criticos
SELECT booking_status_id, code FROM booking.booking_status
WHERE booking_status_id = 5;
SELECT payment_status_id, code FROM payment.payment_status
WHERE payment_status_id = 3;
```

Tabla 10**Resultados esperados de los datos iniciales**

Tabla	Resultado esperado
security.role	3 filas: ADMIN, OPERATOR y CLIENT.
booking.booking_status	6 filas; el identificador 5 corresponde a CANCELLED.
execution.execution_status	5 filas.
payment.payment_status	5 filas; el identificador 3 corresponde a APPROVED.
payment.payment_method_type	4 filas: efectivo, Nequi, Daviplata y transferencia bancaria.
payment.loyalty_movement_type	4 filas.
notification.notification_type	12 filas: 11 ligadas a eventos más SYSTEM_MESSAGE.
customer.vehicle_type, catalog.service_category, booking.service_bay, booking.business_hour	Con filas, según los changesets 102, 103 y 109.

Nota. Fuente: 06-data/models.md y migration-strategy.md. Si una tabla de catálogo está vacía, el changeset de datos iniciales correspondiente no se ejecutó.

11.7 Respaldo y reinicio de la base local

Antes de aplicar cambios de esquema en un entorno con datos que importen, se respalda la base. SQL Server en Docker guarda sus archivos en `/var/opt/mssql/`; el respaldo se genera dentro del contenedor y se copia al anfitrión:

```
sqlcmd -S localhost,1433 -U sa -P "<CONTRASENA_SA>" -C -Q \
"BACKUP DATABASE [<BD>] TO DISK = N'/var/opt/mssql/data/respaldo.bak' WITH INIT"
docker cp <CONTENEDOR_SQLSERVER>:/var/opt/mssql/data/respaldo.bak ./respaldo.bak
```

Para reiniciar por completo un entorno local descartable, se detienen los contenedores y se elimina el volumen de datos. **Esta acción borra todos los datos de la base** y no debe ejecutarse en un entorno con información que se quiera conservar:

```
docker compose down -v
```

La documentación no define una estrategia de respaldo de producción, porque la producción está fuera del alcance de esta versión.

12. Ejecución del backend

12.1 Orden de arranque

La documentación establece que la aplicación no debe depender de un orden de arranque fijo más allá de la disponibilidad de la infraestructura: SQL Server y RabbitMQ deben estar disponibles antes de los servicios que los usan, y cada servicio debe comprobar su disponibilidad en lugar de suponer que otro contenedor está listo por haber iniciado. La secuencia práctica local es la de la Tabla 11.

Tabla 11
Secuencia de arranque del entorno local

Paso	Componente	Condición antes de continuar
1	SQL Server	Acepta conexiones y la base está migrada (sección 11).
2	RabbitMQ	Consola de administración responde (solo si MESSAGING_ENABLED es true).
3	Mailpit	Interfaz web disponible.
4	security-service	Inicia sin errores; emite el JWT.
5	customer-service	Inicia sin errores; lo consulta booking-service.
6	booking-service	Inicia sin errores.
7	operations-service	Inicia sin errores.
8	payment-service	Inicia sin errores.
9	notification-service	Inicia sin errores; crea su cola en RabbitMQ si MESSAGING_ENABLED es true.
10	API Gateway	Responde en su endpoint de salud.

Nota. Fuente: deployment.md, sección 5. Un orden distinto a partir del paso 4 es posible mientras la infraestructura esté lista.

12.2 Modo mínimo sin mensajería

Con `MESSAGING_ENABLED=false` (valor por defecto), los servicios de seguridad y de notificaciones no usan RabbitMQ y el sistema puede ejecutarse con SQL Server únicamente (ADR-011). Este modo es útil para una primera instalación o una máquina con poca memoria, pero no genera notificaciones a partir de eventos de otros servicios. Los servicios de reservas, clientes y pagos siguen necesitándose entre sí por REST.

12.3 Despliegue con Docker Compose (método recomendado)

Desde el directorio de infraestructura, se levanta primero la infraestructura y se espera a que esté lista; luego se levantan los servicios en el orden de la Tabla 11. Los nombres entre ángulos deben tomarse de la salida de `docker compose config --services`.

```
cd ~/lavarapido/lavarapido-infra

# 1-3. Infraestructura
docker compose up -d <SERVICIO_SQLSERVER> <SERVICIO_RABBITMQ> <SERVICIO_MAILPIT>
docker compose ps

# (aplicar migraciones: seccion 11.3)

# 4-9. Servicios de dominio, en orden
docker compose up -d --build security-service
docker compose up -d --build customer-service
docker compose up -d --build booking-service
docker compose up -d --build operations-service
docker compose up -d --build payment-service
docker compose up -d --build notification-service

# 10. Puerta de enlace
docker compose up -d --build <SERVICIO_GATEWAY>
```

La opción *--build* reconstruye la imagen a partir del código actual; se omite cuando solo se quiere reutilizar una imagen ya construida. Si el archivo de Compose declara dependencias con verificación de salud, *docker compose up -d* sin nombres de servicio levanta todo en el orden correcto, y los comandos anteriores pueden resumirse a uno solo.

```
docker compose up -d --build
docker compose ps
```

Figura 7

Contenedores del entorno en ejecución

[ESPACIO RESERVADO PARA LA CAPTURA]

Ubicación/Pantalla: Terminal, lavarapido-infra.

Descripción exacta de la captura: Salida de *docker compose ps* con SQL Server, RabbitMQ, Mailpit, los seis servicios y el gateway en estado running (o healthy), con sus puertos publicados.

12.4 Ejecución de un servicio fuera de Docker (alternativa de desarrollo)

Para depurar un servicio desde un IDE o la terminal, se mantienen en Docker solo la infraestructura y los demás servicios, y se ejecuta el servicio deseado con Maven. Las variables del archivo *.env* deben estar presentes en la sesión; con Bash se cargan así:

```
cd ~/lavarapido/lavarapido-backend/<DIRECTORIO_DEL_SERVICIO>
set -a; source ../../lavarapido-infra/.env; set +a
mvn spring-boot:run
```

En PowerShell, cada variable se define con *\$env:NOMBRE = 'valor'* antes de ejecutar el comando. Si el servicio se ejecuta fuera de Docker, sus direcciones hacia los otros servicios deben apuntar a *localhost* y a los puertos publicados, y no a los nombres internos de la red de Docker. Debe cuidarse que el mismo servicio no esté ejecutándose a la vez en Docker, porque ocuparía el mismo puerto.

12.5 Configuración de RabbitMQ y de correo

Con `MESSAGING_ENABLED=true`, la topología de RabbitMQ se crea cuando inician los servicios: un único exchange de tipo `topic` llamado `carwash.events`, la cola `notification-service.events` enlazada a las claves de enrutamiento que consume, y la cola de mensajes fallidos `notification-service.events.dlq`; los mensajes fallidos se reintentan tres veces antes de pasar a esta última. Con `MAIL_ENABLED=true` y Mailpit, los correos de bienvenida y de recuperación de contraseña no salen a internet: se leen en la interfaz web de Mailpit.

12.6 Registros, detención y actualización

```
docker compose logs -f security-service          # seguir el registro de un servicio
docker compose logs --since 10m booking-service # últimos 10 minutos
docker compose stop                               # detener sin borrar datos
docker compose start                              # reanudar
docker compose down                              # elimina contenedores, conserva volúmenes
docker compose up -d --build <servicio>          # actualizar un servicio tras un cambio
```

Los servicios escriben sus registros en la salida estándar, con marca de tiempo, nivel, nombre del servicio e identificador de correlación. En una instalación sana no deben aparecer trazas de pila durante el arranque. Un registro nunca debe contener contraseñas, tokens ni contenido de comprobantes; si aparecen, se trata de un defecto que debe reportarse.

12.7 Almacenamiento de comprobantes de pago

La documentación indica que el almacenamiento de los comprobantes de pago es una decisión de implementación aún pendiente, por lo que este manual no prescribe una ruta de carpeta ni un formato. Si el servicio de pagos guarda archivos en disco, debe definirse un volumen persistente y respaldarse junto con la base de datos (Anexo B).

13. Ejecución del frontend

13.1 Aplicación web (Angular)

La aplicación web consume únicamente el API Gateway. Antes de iniciarla, el gateway debe responder (sección 16) y la dirección del gateway debe estar configurada en la aplicación (sección 10.6).

```
cd ~/lavarapido/Front-end-proyecto-web
npm run          # lista los scripts disponibles en package.json
npm start       # o: npx ng serve
```

El servidor de desarrollo de Angular publica la aplicación, por defecto, en `http://localhost:4200`. El script exacto (`start`, `serve` o equivalente) se toma de la salida de `npm run`. Si el navegador bloquea las peticiones al gateway con un error de CORS, el origen `http://localhost:4200` debe estar permitido en la configuración de seguridad del backend (sección 17).

Para generar la versión de producción de la aplicación web, se ejecuta el script de compilación del repositorio (habitualmente `npm run build`), que deja los archivos estáticos en un directorio de salida. La publicación de esos archivos en un servidor web queda fuera del alcance de este manual, porque la documentación no define un entorno de producción.

Figura 8

Aplicación web en ejecución

[ESPACIO RESERVADO PARA LA CAPTURA]

Ubicación/Pantalla: Navegador, dirección `http://localhost:4200`.

Descripción exacta de la captura: Pantalla de bienvenida de la aplicación con la barra superior, el selector de idioma y los botones Iniciar sesión y Registrarse, junto con la barra de direcciones visible.

13.2 Aplicación móvil (React Native con Expo)

```
cd ~/lavarapido/LavadoVehicular-Movil
npx expo start
```

El comando abre el servidor de desarrollo (Metro, puerto 8081 por defecto) y muestra un código QR. Para una prueba rápida de la interfaz, puede abrirse en un emulador de Android desde la propia consola de Expo (tecla `a`) o en un teléfono con la aplicación Expo Go, escaneando el código QR en la misma red local. Debe recordarse que la dirección del gateway para el emulador es `10.0.2.2` y para un teléfono físico, la IP local del equipo.

Notificaciones push. Expo Go no recibe notificaciones push remotas. Para probarlas, se necesita una compilación de desarrollo con EAS, y en el backend debe activarse `PUSH_ENABLED=true` (ADR-011). Un teléfono iOS requeriría además una cuenta de desarrollador de Apple, por lo que el alcance se limita a Android.

```
npx eas-cli login
```

```
npx eas-cli build --profile development --platform android
```

Figura 9**Servidor de desarrollo de Expo**

[ESPACIO RESERVADO PARA LA CAPTURA]

Ubicación/Pantalla: Terminal, directorio LavadoVehicular-Movil.

Descripción exacta de la captura: Salida de npx expo start con el código QR, la dirección de Metro y las opciones de teclado para abrir en Android.

13.3 Maquetas estáticas (opcional)

El repositorio de documentación incluye maquetas HTML de las pantallas en *12-ux-ui/mockup*. No requieren backend: se abren directamente en el navegador, y sirven como prueba visual rápida de la interfaz esperada. No deben confundirse con la aplicación real.

14. Acceso al sistema

Una vez iniciados todos los componentes, las personas acceden al sistema a través de la aplicación web o de la aplicación móvil, y el personal técnico accede además a las interfaces de apoyo de la Tabla 12. Todo acceso de los clientes a la API pasa por el gateway.

Tabla 12

Puntos de acceso del entorno local

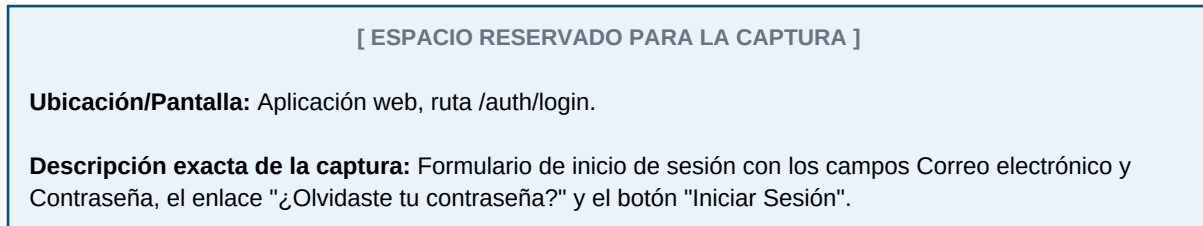
Acceso	Dirección	Quién lo usa
Aplicación web	http://localhost:4200	Clientes, operarios y administradores.
Aplicación móvil	Expo Go o compilación de desarrollo	Clientes y operarios.
API Gateway	http://localhost:<PUERTO_GATEWAY> (8080 en la plantilla)	Aplicaciones cliente y pruebas de API.
Mailpit (correo de desarrollo)	http://localhost:8025	Personal técnico: lectura del código de recuperación y del correo de bienvenida.
Consola de RabbitMQ	http://localhost:15672	Personal técnico: colas, exchange y mensajes fallidos.
SQL Server	localhost,1433	Personal técnico: sqlcmd o cliente gráfico.

Nota. Los puertos son los valores por defecto de cada herramienta y deben confirmarse en el archivo de Compose (Anexo B).

14.1 Primer acceso a la aplicación web

- 1 Abrir un navegador actualizado (Chrome, Firefox o Edge) en *http://localhost:4200*.
- 2 Comprobar que se muestra la pantalla de bienvenida con los botones **Iniciar sesión** y **Registrarse**.
- 3 Hacer clic en **Iniciar sesión** y comprobar que aparece el formulario con los campos de correo electrónico y contraseña.
- 4 Iniciar sesión con una cuenta de prueba (sección 15). El sistema muestra el panel que corresponde al rol de la cuenta.

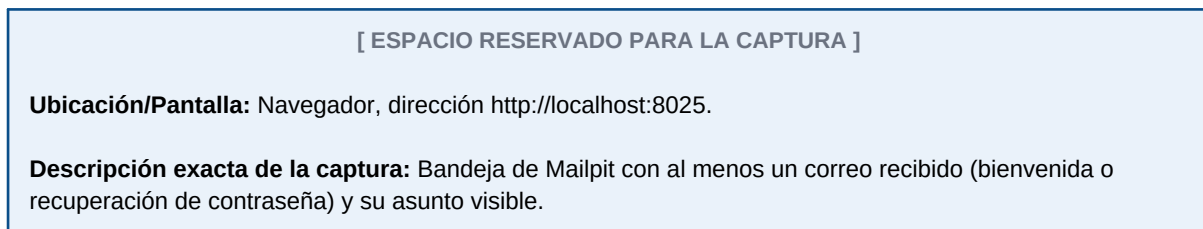
Figura 10
Pantalla de inicio de sesión



14.2 Interfaces de apoyo

La interfaz de Mailpit muestra los correos que el sistema "envía" en local, incluido el código de seis dígitos de la recuperación de contraseña, que vence a los 15 minutos. La consola de RabbitMQ (con las credenciales definidas en el archivo `.env`) permite ver el exchange `carwash.events`, las colas y los mensajes pendientes o fallidos. Ninguna de las dos interfaces debe exponerse fuera del equipo local.

Figura 11
Interfaz web de Mailpit



Nota. No mostrar direcciones de correo reales.

15. Usuario de prueba

Los datos iniciales del sistema (changesets 101 a 109) cargan roles, catálogos, estados, bahías y horarios, pero **no incluyen cuentas de usuario**. Por esa razón, el primer usuario de prueba se crea mediante el propio flujo de registro del sistema. La política de seguridad prohíbe publicar contraseñas reales en este manual y exige que las credenciales de demostración nunca lleguen a un despliegue público; por eso aquí se describen las reglas y el procedimiento, no credenciales concretas.

15.1 Reglas para las cuentas

Tabla 13

Reglas de los datos de una cuenta de prueba

Dato	Regla
Correo electrónico	Debe terminar en @gmail.com; no admite espacios. Cada correo admite una sola cuenta.
Nombre completo	Mínimo 3 caracteres.
Teléfono	Celular colombiano de 10 dígitos que empieza por 3.
Contraseña	Mínimo 8 caracteres, con al menos una mayúscula, un número y un carácter especial.
Bloqueo	Cinco intentos fallidos en 15 minutos bloquean la cuenta durante 15 minutos.
Vigencia de la sesión	El token de acceso dura una hora; después se vuelve a iniciar sesión.

Nota. Fuente: reglas de validación del registro y política de seguridad del proyecto.

15.2 Creación de un cliente de prueba

- 1 Abrir la aplicación web y hacer clic en **Registrarse**.
- 2 Diligenciar nombre, un correo de Gmail **de prueba**, un teléfono con el formato válido y una contraseña que cumpla la política. Se recomienda registrar la contraseña en un gestor de contraseñas del equipo y no en un documento.
- 3 Hacer clic en **Registrarse**; el sistema lleva a la pantalla de inicio de sesión.
- 4 Abrir Mailpit (<http://localhost:8025>) y comprobar que llegó el correo de bienvenida, siempre que `MAIL_ENABLED` sea `true`.
- 5 Iniciar sesión con la cuenta creada y comprobar que se muestra el panel del cliente.

15.3 Cuentas de operario y de administrador

El rol de la cuenta registrada por el flujo público es CLIENT. La documentación indica que los operarios reciben su cuenta de parte del administrador, pero **no describe cómo se crea la primera cuenta administradora** ni si existe un procedimiento o un script para ello. Este

procedimiento debe definirse y documentarse antes de la entrega (Anexo B). Mientras tanto, la forma de ejecutar de principio a fin el flujo de negocio con los tres roles es el archivo de solicitudes *http/flujo-demo.http* del repositorio *lavarapido-backend*, que la matriz de trazabilidad identifica como la prueba de flujo completo ejecutada a mano; se abre con la extensión REST Client de Visual Studio Code o se importa en Postman.

Toda cuenta de demostración debe eliminarse o cambiar su contraseña antes de exponer el entorno fuera del equipo local.

16. Verificación de instalación

La verificación se hace por capas: infraestructura, base de datos, mensajería, API y flujo funcional. La documentación solicita comprobar la conectividad con la base de datos y con RabbitMQ, la autenticación con JWT, la validación entre servicios, la creación de reservas con manejo de conflictos y la revisión de pagos. La Tabla 14 resume las pruebas, que se detallan a continuación.

Tabla 14

Lista de verificación de la instalación

N.º	Comprobación	Resultado esperado
1	Contenedores en ejecución	Todos en estado running o healthy.
2	Conexión a SQL Server	SELECT @@VERSION devuelve la versión.
3	Nueve esquemas y datos iniciales	Consultas de la sección 11.6 con los resultados esperados.
4	RabbitMQ	Exchange carwash.events y colas presentes (con mensajería activa).
5	Mailpit	Interfaz web disponible.
6	Salud del gateway	Respuesta 200 con estado ok (o degraded si falta un servicio no crítico).
7	Autenticación en el borde	Una ruta protegida sin token responde 401.
8	Aplicación web	La pantalla de bienvenida y el inicio de sesión se visualizan.
9	Registro e inicio de sesión	Se crea una cuenta y se ingresa al panel del cliente.
10	Recuperación de contraseña	El código llega a Mailpit y permite cambiar la contraseña.
11	Flujo de reserva	Se registra un vehículo y se crea una reserva.
12	Notificaciones	La reserva confirmada genera una notificación dentro del sistema.
13	Auditoría	audit.audit_log registra los cambios con el usuario autor.
14	Persistencia	Los datos siguen presentes al reiniciar los contenedores.

Nota. Elaboración propia con base en las verificaciones de integración de deployment.md.

16.1 Infraestructura y base de datos

```
docker compose ps
sqlcmd -S localhost,1433 -U sa -P "<CONTRASENA_SA>" -C -Q "SELECT @@VERSION"
```

A continuación se ejecutan las consultas de la sección 11.6. Si los resultados no coinciden, no se continúa con las pruebas siguientes.

16.2 Mensajería y correo

Con mensajería activa, se verifica que el exchange y las colas existen. Las credenciales son las del archivo `.env`; el host virtual por defecto es `%2F` (la barra diagonal codificada), salvo que el entorno defina otro.

```
curl -s -u <USUARIO_RABBIT>:<CONTRASENA_RABBIT> \
  http://localhost:15672/api/exchanges/%2F/carwash.events
docker compose exec <SERVICIO_RABBITMQ> rabbitmqctl list_queues name messages
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8025/
```

La primera orden debe devolver los datos del exchange (tipo *topic*); la segunda, las colas con sus mensajes (la cola *notification-service.events.dlq* debe estar en cero); la tercera, el código HTTP 200 de Mailpit.

16.3 API Gateway

```
curl -i http://localhost:<PUERTO_GATEWAY>/health
curl -i http://localhost:<PUERTO_GATEWAY>/api/v1/bookings/me
```

La primera orden debe devolver el estado 200 con el estado general del sistema (*ok* o *degraded*), o 503 si un servicio crítico no está disponible, según el contrato del gateway. La segunda, que consulta una ruta protegida sin token, debe devolver 401, lo que demuestra que el gateway rechaza en el borde las peticiones sin JWT.

Figura 12

Respuesta del endpoint de salud del gateway

[ESPACIO RESERVADO PARA LA CAPTURA]

Ubicación/Pantalla: Terminal, comando curl contra el gateway.

Descripción exacta de la captura: Salida de curl -i con el código de estado 200 y el cuerpo con el estado de los servicios.

16.4 Flujo funcional mínimo

- 1 Registrar un cliente de prueba y comprobar el correo de bienvenida en Mailpit.
- 2 Probar la recuperación de contraseña: solicitar el código, leerlo en Mailpit y cambiar la contraseña.
- 3 Iniciar sesión y registrar un vehículo.
- 4 Crear una reserva con ese vehículo. Un turno libre debe confirmarse; uno ocupado debe proponer horarios alternativos.

- 5 Con mensajería activa, comprobar que la reserva confirmada produjo una notificación en el módulo de notificaciones del cliente.
- 6 Reiniciar los contenedores (*docker compose restart*) y comprobar que los datos creados siguen presentes al volver a ingresar.

16.5 Verificación de la auditoría

Los disparadores de auditoría leen el usuario del contexto de sesión de SQL Server, que cada servicio debe establecer al inicio de cada petición. Si no se establece, los cambios quedan sin autor y no se produce ningún error visible. Tras crear la reserva de la prueba anterior, se consulta el registro más reciente:

```
SELECT TOP (5) * FROM audit.audit_log ORDER BY 1 DESC;
```

El campo que identifica al autor del cambio no debe estar vacío en los registros generados por las acciones del paso anterior. Si está vacío, el servicio correspondiente no está estableciendo el contexto de sesión y debe informarse al equipo de desarrollo (sección 17).

Figura 13

Verificación del registro de auditoría

[ESPACIO RESERVADO PARA LA CAPTURA]

Ubicación/Pantalla: Cliente SQL o sqlcmd, consulta a audit.audit_log.

Descripción exacta de la captura: Resultado de la consulta con las filas más recientes, mostrando la tabla modificada, el tipo de operación y el campo del usuario autor con valor.

17. Errores comunes y soluciones

La Tabla 15 reúne los problemas más probables según la arquitectura documentada. Ante cualquier error, el primer paso es consultar el registro del servicio afectado con *docker compose logs --tail 100 <servicio>*.

Tabla 15**Errores frecuentes, causas probables y soluciones**

Error o síntoma	Causa probable	Solución
Un puerto está ocupado (address already in use)	Otro proceso usa el puerto publicado.	Identificar el proceso (<i>netstat -ano</i> en Windows, <i>lsof -i :PUERTO</i> en Linux o macOS) y detenerlo, o cambiar el puerto en <i>.env</i> o en Compose.
El contenedor de SQL Server se detiene al iniciar	Contraseña de sa débil, licencia no aceptada o menos de 2 GB de memoria.	Revisar <i>docker compose logs</i> ; usar una contraseña que cumpla la política, <i>ACCEPT_EULA=Y</i> y aumentar la memoria de Docker.
Login failed for user 'sa'	Contraseña distinta de la del primer arranque del volumen.	La contraseña se fija al crear el volumen: usar la original o eliminar el volumen (<i>docker compose down -v</i>) en un entorno descartable.
sqlcmd no se conecta a localhost:1433	Sintaxis o certificado.	Usar <i>-S localhost,1433</i> (con coma) y agregar <i>-C</i> para aceptar el certificado autofirmado.
Error de certificado TLS en Liquibase o en los servicios	Certificado autofirmado del contenedor.	Solo en local: <i>trustServerCertificate=true</i> en la URL JDBC.
Liquibase: checksum mismatch	Se editó un changeset ya ejecutado.	No editarlo. Revertir el cambio en el archivo y registrar la modificación en un changeset nuevo. En un entorno descartable, recrear la base.
Liquibase: no puede obtener el bloqueo	Una ejecución anterior se interrumpió y dejó el bloqueo.	Confirmar que no hay otra ejecución y usar <i>liquibase release-locks</i> .
Faltan los disparadores tr_touch_	El changeset 013 no se ejecutó, o se ejecutó antes que las tablas.	Verificar el orden del changelog maestro. Una tabla nueva creada después del 013 debe crear su propio disparador.
Fallan las reglas de reserva o de pago tras resembrar	Los identificadores fijos de los catálogos cambiaron (CANCELLED = 5, APPROVED = 3).	Resembrar con <i>IDENTITY_INSERT</i> y los identificadores originales; verificar con las consultas de la sección 11.6.
401 en toda petición autenticada	Token vencido (una hora) o secreto del JWT distinto entre servicios.	Volver a iniciar sesión y comprobar que el secreto es idéntico en todos los servicios.
503 CUSTOMER_SERVICE_UNAVAILABLE al reservar	customer-service caído o con dirección o tiempo de espera mal configurados.	Revisar <i>CUSTOMER_SERVICE_URL</i> , <i>CUSTOMER_SERVICE_TIMEOUT</i> y el estado del servicio.
Los servicios no se conectan a RabbitMQ	MESSAGING_ENABLED activo pero RabbitMQ caído o credenciales erróneas.	Iniciar RabbitMQ, revisar sus credenciales o poner <i>MESSAGING_ENABLED</i> en false.

Error o síntoma	Causa probable	Solución
No llegan notificaciones	MESSAGING_ENABLED en false, evento sin identificador del destinatario o mensajes en la cola fallida.	Activar la mensajería y revisar la cola <i>notification-service.events.dlq</i> .
No llegan correos a Mailpit	MAIL_ENABLED en false o servidor y puerto SMTP erróneos.	Configurar MAIL_* con el servidor de Mailpit y el puerto 1025.
Las notificaciones push no se entregan	PUSH_ENABLED en false, uso de Expo Go o token desactivado.	Activar PUSH_ENABLED y usar una compilación de desarrollo con EAS.
Error de CORS en el navegador	El origen de la aplicación web no está permitido.	Agregar <i>http://localhost:4200</i> a los orígenes permitidos del backend.
Pantalla en blanco en la aplicación web	Dirección del gateway errónea o error de compilación.	Revisar la consola del navegador, la dirección del gateway configurada y el registro de <i>npm start</i> .
La app móvil no alcanza el backend	Se usa localhost desde el emulador o el teléfono está en otra red.	Usar 10.0.2.2 en el emulador, o la IP local del equipo en la misma red en un teléfono físico.
UnsupportedClassVersionError	El JDK instalado es anterior a la versión 21.	Instalar JDK 21 y verificar con <i>java -version</i> .
npm ERESOLVE o fallos de compilación	Versión de Node.js distinta de la esperada.	Usar la versión de Node.js indicada por el repositorio y reinstalar con <i>npm ci</i> .
Docker consume toda la memoria en Windows	WSL 2 sin límite de memoria.	Limitar la memoria de WSL 2 en el archivo <i>.wslconfig</i> y asignar al menos 5 GB a Docker.
Las horas de las reservas aparecen desfasadas	Zona horaria mal configurada (el sistema almacena en UTC).	Revisar BOOKING_ZONE y la conversión a la hora de Colombia en el cliente.
La cuenta aparece bloqueada	Cinco intentos fallidos en 15 minutos.	Esperar 15 minutos, recuperar la contraseña o reiniciar el servicio de seguridad (los contadores están en memoria).
El autor de los cambios aparece vacío en audit_log	El servicio no establece el contexto de sesión (SESSION_CONTEXT).	Informar al equipo de desarrollo: debe resolverse en un filtro común de cada servicio.

Nota. Elaboración propia con base en las decisiones y los riesgos documentados.

18. Glosario

Tabla 16**Glosario de términos de instalación**

Término	Definición
API Gateway	Punto único de entrada externo; enruta las peticiones y valida el JWT en el borde.
Changelog	Archivo de Liquibase que describe cambios de la base de datos.
Changeset	Unidad de cambio de un changelog, identificada por identificador, autor y ruta.
Compose	Herramienta que define y ejecuta varios contenedores con un archivo de configuración.
Contenedor	Proceso aislado que ejecuta una aplicación con sus dependencias.
CORS	Mecanismo del navegador que restringe las peticiones entre orígenes distintos.
DLQ (cola de mensajes fallidos)	Cola donde se conservan los mensajes que no pudieron procesarse tras los reintentos.
Exchange	Elemento de RabbitMQ que recibe mensajes y los enruta a colas.
Imagen	Plantilla de solo lectura a partir de la cual se crea un contenedor.
JDK	Kit de desarrollo de Java; el proyecto usa la versión 21.
JWT	Token firmado que identifica al usuario y su rol; en el sistema dura una hora.
Liquibase	Herramienta de versionado y migración del esquema de base de datos.
Mailpit	Servidor SMTP de desarrollo que captura los correos sin enviarlos.
Migración	Cambio versionado del esquema o de los datos de la base.
RabbitMQ	Bróker de mensajes que transporta los eventos de dominio.
SESSION_CONTEXT	Mecanismo de SQL Server para pasar el usuario actual a los disparadores de auditoría.
Volumen	Almacenamiento persistente de Docker que sobrevive a la recreación del contenedor.
WSL 2	Subsistema de Windows para Linux que usa Docker Desktop.

Nota. Elaboración propia con base en la documentación del proyecto.

19. Anexos

Anexo A. Lista de figuras pendientes de captura

Antes de la entrega, el equipo debe reemplazar cada espacio reservado del documento por una captura real, nítida y ordenada según el flujo de instalación. Las capturas no deben contener contraseñas, secretos, cuentas ni datos personales reales.

Anexo B. Información por confirmar antes de la entrega

La Tabla 17 reúne los datos que este manual no pudo fijar porque dependen del repositorio de infraestructura o no están en la documentación. Cada elemento debe resolverse y sustituirse en el cuerpo del manual.

Tabla 17
Información pendiente de confirmar

Tema	Qué falta o qué se contradice	Dónde se usa
Direcciones de los repositorios	El material analizado no incluye las URL.	Sección 8
Archivo de Compose y nombres de servicio	No se dispone del repositorio de infraestructura; los nombres de servicio de SQL Server, RabbitMQ, Mailpit y gateway son marcadores.	Secciones 11 y 12
Puertos	Solo se conocen los valores por defecto de las imágenes; el puerto de cada servicio y el del gateway (8080 es de una plantilla) no están documentados.	Secciones 6, 14 y 16
Nombres de las variables de entorno	Se confirman CUSTOMER_SERVICE_URL, CUSTOMER_SERVICE_TIMEOUT, BOOKING_ZONE, MESSAGING_ENABLED, PUSH_ENABLED, MAIL_ENABLED y MAIL_*. Faltan los de base de datos, JWT y RabbitMQ.	Sección 10
Nombre de la base de datos	No está fijado.	Sección 11
Quién ejecuta Liquibase y dónde residen los changelogs	La estrategia define un changelog maestro único, pero no indica si lo ejecuta un paso de despliegue o el arranque de un servicio, ni el repositorio donde reside.	Secciones 8 y 11
Cuentas de base de datos por servicio	No se documentan los nombres de los roles del changeset 001 ni el procedimiento para crear los inicios de sesión.	Sección 11.5
Script único y tablas nuevas	La ADR-011 señala que lavarapido-6-services-sqlserver.sql debe incluir device_token y processed_event.	Sección 11.4
Creación de la primera cuenta de administrador y de operarios	Los datos iniciales no cargan usuarios y no hay un procedimiento documentado.	Sección 15
Herramienta de construcción y versiones	No se indica si se usa Maven o Gradle ni las versiones de Node.js, Angular y Expo.	Secciones 7 y 9
Salud de cada servicio	Solo se documenta el endpoint del gateway; los de los servicios deben tomarse del código.	Sección 16
Almacenamiento de comprobantes de pago	Decisión pendiente; no hay volumen ni ruta definidos.	Sección 12.7
Lenguaje de los servicios	La documentación afirma Java 21 y Spring Boot en los seis servicios; confirmar con el código.	Secciones 7 y 12
Documentos plantilla	local-setup.md y .env.example de la documentación mencionan PostgreSQL, Redis y puertos genéricos; se ignoraron.	Sección 4.1

Nota. Elaboración propia a partir del análisis de la documentación del proyecto.

Anexo C. Resumen de comandos de instalación

Secuencia resumida para un entorno local con Docker y Liquibase. Los marcadores entre ángulos deben sustituirse antes de ejecutar los comandos.

```
# 1. Obtener el proyecto
mkdir -p ~/lavarapido && cd ~/lavarapido
git clone <URL_REPOSITORIO_INFRA> lavarapido-infra
git clone <URL_REPOSITORIO_BACKEND> lavarapido-backend
git clone <URL_REPOSITORIO_WEB> Front-end-proyecto-web
git clone <URL_REPOSITORIO_MOVIL> LavadoVehicular-Movil

# 2. Configurar el entorno
cd lavarapido-infra && cp .env.example .env # editar .env con valores propios
git check-ignore -v .env

# 3. Infraestructura y base de datos
docker compose up -d <SERVICIO_SQLSERVER> <SERVICIO_RABBITMQ> <SERVICIO_MAILPIT>
sqlcmd -S localhost,1433 -U sa -P "<CONTRASENA_SA>" -C -Q \
    "IF DB_ID(N'<BD>') IS NULL CREATE DATABASE [<BD>];"
liquibase validate && liquibase update

# 4. Servicios y gateway
docker compose up -d --build
docker compose ps

# 5. Clientes
cd ../Front-end-proyecto-web && npm ci && npm start
cd ../LavadoVehicular-Movil && npm ci && npx expo start

# 6. Verificacion
curl -i http://localhost:<PUERTO_GATEWAY>/health
```

Anexo D. Lista de chequeo de seguridad previa a compartir el entorno

Tabla 18

Controles de seguridad del entorno

Control	Cumple
El archivo .env no está versionado y tiene permisos restringidos	Sí / No
El secreto del JWT tiene al menos 32 caracteres y no es el de demostración	Sí / No
Las contraseñas de sa, de los servicios y de RabbitMQ fueron cambiadas	Sí / No
Los servicios no usan la cuenta sa para conectarse	Sí / No
Solo el gateway está expuesto fuera de la red de Docker	Sí / No
Mailpit y la consola de RabbitMQ no son accesibles desde fuera del equipo	Sí / No
Las cuentas de demostración fueron eliminadas o con contraseña cambiada	Sí / No
Se usa HTTPS con TLS 1.2 o superior si el entorno sale del equipo local	Sí / No
Los registros no contienen contraseñas, tokens ni comprobantes	Sí / No

Nota. Elaboración propia con base en la política y las reglas de seguridad del proyecto.

Anexo E. Lista de chequeo de entrega

Tabla 19***Lista de chequeo del manual de instalación***

Criterio	Cumple
Portada completa y versión	Sí / No
Tabla de contenido con páginas actualizadas	Sí / No
Objetivo y alcance explicados	Sí / No
Los comandos coinciden con el repositorio de infraestructura real	Sí / No
Todos los marcadores entre ángulos fueron sustituidos	Sí / No
Todas las figuras reemplazadas por capturas reales	Sí / No
Pasos claros y verificados en una instalación limpia	Sí / No
Sin credenciales, claves ni datos sensibles	Sí / No
Exportado en PDF con el nombre Manual_instalacion_NombreProyecto_Ficha.pdf	Sí / No

Nota. Adaptada de la lista de chequeo de la guía ADSO (SENA, 2026).

Referencias

- Equipo desarrollador. (2026). *Documentación del Sistema de Gestión de Lavado Vehicular (Vehicle Washing)* [Repositorio de documentación del proyecto]. Programa Tecnólogo en Análisis y Desarrollo de Software, SENA.
- Humble, J., y Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.
- Microsoft. (s. f.). *Microsoft SQL Server - Ubuntu based images* [Imagen de contenedor]. Docker Hub. <https://hub.docker.com/r/microsoft/mssql-server>
- Servicio Nacional de Aprendizaje [SENA]. (2026). *Guía para elaborar manuales de software: Manual de usuario, manual técnico, manual de instalación y manual de administrador*. Tecnólogo en Análisis y Desarrollo de Software.
- Wiggins, A. (2017). *The twelve-factor app: III. Config*. <https://12factor.net/config>